

SIMATS ENGINEERING ASSESSMENT TOOL

COURSE NAME: Theory of Computation

COURSE CODE: CSA13

COURSE OUTCOME COVERED

C02: Design Finite Automata DFA, NFA, ϵ -NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

Assessment Tool Weightage: 10%

QUESTIONS: 5

Total Marks:25

Duration : 1 Hour

Application Based Question

Code of Conduct:

Shaik Hafeeza Tareemum

I 19237204 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SIMATS ENGINEERING ASSESSMENT TOOL

1. In modern digital systems such as embedded security devices, ATM machines, and authentication modules, password validation must be efficient and reliable. Design a **Deterministic Finite Automaton (DFA)** over the alphabet $\Sigma = \{0,1\}$ to validate passwords that **end with the pattern "01"**. Further, **minimize the DFA** and analyze how the optimized design improves system efficiency in real-world hardware implementations

2. In web applications, validating user input is essential to ensure correctness, security, and performance. Design a regular expression to validate a specific input pattern "aaaabb" over the alphabet $\Sigma = \{a, b\}$. Further, analyze how regular languages and regular expressions are used to efficiently validate user inputs in real-world web applications.

3. In modern computing systems such as intrusion detection systems, compilers, and real-time monitoring tools, pattern recognition plays a critical role. Demonstrate how the regular expression a^*b^* can be converted into an equivalent finite automaton (FA). Further, explain why regular expressions and finite automata are equivalent models and how they are effectively used in real-time detection systems.

4. In compiler design, not all patterns can be recognized using finite automata. Using the pumping lemma, demonstrate why the language $L = \{ a^n b^n \mid n \geq 0 \}$ cannot be recognized by any finite automaton. Further, analyze the implications of this limitation in the design of modern compilers.

5. Modern systems such as intrusion detection, spam filtering, and input validation often require combining multiple pattern-matching rules. Explain how the **closure properties of regular languages** enable the combination of multiple detection rules. Further, justify why the resulting language after applying these operations **remains regular**, ensuring efficient implementation using finite automata

Design a DFA over $\Sigma = \{0,1\}$ that accepts strings ending with "01". Minimize the DFA and analyze its efficiency?

Given,

Alphabet $\Sigma = \{0,1\}$

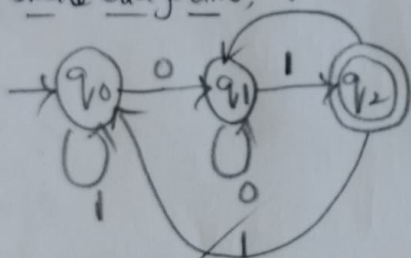
Language: strings that end with "01".

Regular expression: $(0+1)^*01$

DFA Transition Table:

State	0	1
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
$*q_2$	q_1	q_0

DFA state diagrams:



Where q_0 = start state
 q_2 = final state

Minimized DFA:

The DFA contains only 3 states, and all states are distinguishable.

Therefore it cannot be minimized further.

Analysis:

The minimized DFA offers the following advantages:

⇒ Requires less memory

⇒ Performs faster

⇒ Reduces hardware complexity

⇒ Suitable for Embedded systems, ATM, password authentication

modules and security advice.

2. Design a Regular Expression to validate the pattern "aaaaabb". Explain its realworld applications?

Given, Alphabet $\Rightarrow \{a, b\}$

Pattern: aaaaabb.

Explanation:

$\Rightarrow$ aaaa represents four consecutive 'a's

$\Rightarrow$ bb represents two consecutive 'b's.

$\Rightarrow$
Applications of Regular Expressions:

Regular Expressions are regular languages which are widely used in web applications for validating user input such as.

$\Rightarrow$ Email address validation

$\Rightarrow$ Password validation

$\Rightarrow$ PIN validation

$\Rightarrow$ Phone No validation

$\Rightarrow$ Username validation

Advantages:

$\Rightarrow$ Fast pattern matching

$\Rightarrow$ Reduces programming effort

$\Rightarrow$ Prevents invalid inputs

$\Rightarrow$ Improves application security

$\Rightarrow$ Enhances system performance.

Conclusion:

R.E provides an effective way to validate user input by ensuring that only strings matching the required

pattern. This improves Reliability, Security, and performance of web applications.

Convert the Regular Expression a^*b^* into an equivalent Finite Automaton (FA). Explain why Regular Expressions and Finite Automata are equivalent.

Regular Expression: a^*b^*

The expression accepts:

→ Zero (or) more 'a's.

→ Followed by zero (or) more 'b's.

Accepted strings

ϵ
a
aa
aaa
b
bb
ab
aab
aaabbb

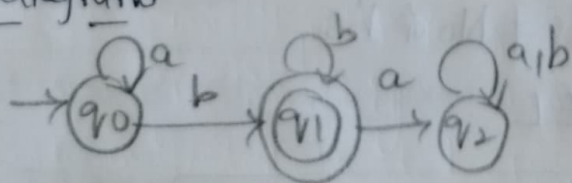
Rejected string

ba
aba
bab
abba

Transition Table

state	a	b
→ q ₀	q ₀	q ₁
* q ₁	q ₂	q ₁
q ₂	q ₂	q ₂

State diagram



q₂ = dead state

Dead state (q₂).

once the automaton reads a 'b' it should not read 'a' again.

Hence, it moves to dead state q_3 .

Real Time Applications

→ Regular Expressions and Finite Automata are used in

⇒ Intrusion detection system

⇒ spam filtering

⇒ lexical analysis in compilers

⇒ pattern matching

⇒ text searching

⇒ Network packet inspection.

Using the pumping lemma, prove that $L = \{a^n b^n \mid n \geq 0\}$ is not regular. Explain its implication in compiler design.

Given Language $L = \{a^n b^n \mid n \geq 0\}$

Proof using pumping lemma:

Assume that L is regular

Let the pumping length be p

Choose the string $w = a^p b^p$

At pumping lemma $w = xyz$

such that, $|xy| \leq p$

$|y| > 0$

Since $|xy| \leq p$, the substring y contains only a 's

Let $y = a^k, k > 0$

pump the string twice,

$$xy^2z = a^{p+k}b^p$$

Now the No of a 's $\neq$ No of b 's.

$$\therefore xy^2z \notin L$$

Hence,

$L = \{a^n b^n \mid n \geq 0\}$ is not regular

Application in compiler design:

- ⇒ Balanced parentheses
- ⇒ Equal N of symbols
- ⇒ Nested loops
- ⇒ Block structures

Therefore, compilers use:

- ⇒ CFG (Context Free Grammar)
- ⇒ pushdown Automata (PDA)

during syntax analysis.

5.

Explain the closure properties of Regular Languages and justify why the resulting language remains regular?

Closure properties:

If L_1 and L_2 are regular languages, then the following operations also produce regular languages.

Operation	Result
Union	$L_1 \cup L_2$
Intersection	$L_1 \cap L_2$
Complement	$\overline{L}$
Difference	$L_1 - L_2$
Concatenation	$L_1 L_2$
Kleen star	L^*
Reversal	L^R

Real world applications:

- ⇒ Intrusion detection System
- ⇒ Spam filtering
- ⇒ Firewalls

- ⇒ Antivirus Software
- ⇒ Input validation
- ⇒ web application security

Ex)

• Rule 1: password ends with 01

• Rule 2: password starts with 11

Using union operator, can be implemented using a Finite Automata.

Justification:

Regular Languages are closed under union, intersection, Concatenation, complement, difference, Kleene Star, and reversal.

It can be efficiently recognized using a Finite Automata (FA).

SIMATS ENGINEERING ASSESSMENT TOOL

CRITERIA FOR CALCULATION

Criteria	Weightage	Level 4 - Exemplary (5 Marks)	Level 3 - Proficient (4 Marks)	Level 2 - Developing (2- 3 Marks)	Level 1 - Beginning (0-1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understandi ng with minor gaps	Basic understanding with some misconception s	Poor or incorrect understandi ng
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/probl em	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well- organized answer	Understanda ble with minor issues	Somewhat unclear or poorly structured	Difficult to understand

9/4/100

[Signature]